



RWA TOKENIZATION PLATFORM

FINAL PROJECT

FULL-STACK DECENTRALIZED PROTOCOL

- **LENARA SYMBATKYZY — GOVERNANCE, FRONTEND, DEVOPS, DOCUMENTATION**
- **AISHABIBI KURALBAY — PROTOCOL CORE, SMART CONTRACTS, SECURITY**

BLOCKCHAIN TECHNOLOGIES 2

2026



Title: What We Built

Text:

A production-grade RWA (Real-World Asset)

Tokenization Platform that enables:

- Issuers to mint asset-backed tokens representing real collateral
- Users to deposit into yield vaults and borrow against assets
- Community to govern all parameters via DAO

```
[Users] → [React Frontend] → [The Graph Subgraph]
                        ↓
[Chainlink Oracles] ← → [Smart Contracts on Arbitrum Sepolia]
                        ↓
                    [Governance DAO]
```

Key Metrics:

- ✓ 16 Smart Contracts
- ✓ 62 Automated Tests (94% Coverage)
- ✓ Deployed & Verified on Arbitrum Sepolia
- ✓ 0 High / 0 Medium Slither Findings
- ✓ Full CI/CD Pipeline

Architecture Diagram (insert C4 diagram):

PROJECT OVERVIEW

SMART CONTRACT ARCHITECTURE

```
// AssetToken.sol - Role-gated minting
function mint(address to, uint256 amount) external onlyRole(ISSUER_ROLE) {
    _mint(to, amount);
}
```

Contract	Standard	Purpose
AssetToken	ERC-20 + AccessControl	Role-gated minting for issuers
AssetCertificate	ERC-721 + AccessControl	Unique asset representation
RwaYieldVault	ERC-4626 + Ownable	Yield generation for depositors
RwaLendingPool	UUPS Upgradeable	Borrow/lend against collateral
ChainlinkPriceOracle	Oracle Adapter	Price feeds with staleness check
VaultFactory	Factory (CREATE+CREATE2)	Deterministic vault deployment
GovernanceToken	ERC20Votes + ERC20Permit	Voting power + gasless delegation
RWAGovernor	OZ Governor	Full proposal lifecycle
RWATimelock	TimelockController	2-day execution delay

Title: Contract Architecture — Protocol Core

Text:

All contracts built from scratch, no forks except OpenZeppelin libraries.

LENDING POOL — CORE DEFI PRIMITIVE

Title: Custom Lending Pool — Built From Scratch

Text:

Our lending pool is the most complex component — fully custom implementation supporting:

- Deposit asset-backed tokens as collateral
- Borrow stablecoins with health factor check
- Repay debt with partial/full repayment
- Liquidation when health factor < 1
- UUPS upgradeable + Pausable + ReentrancyGuard

Security Patterns Applied:

- Checks-Effects-Interactions on all state changes
- SafeERC20 for all token transfers
- ReentrancyGuard on deposit/borrow/repay/liquidate

```
function liquidate(address borrower, address collateralToken)
    external nonReentrant {
        require(healthFactor(borrower) < 1e18, "health factor not below 1");
        // CEI: Update state before external call
        uint256 collateralToSeize = loans[borrower][collateralToken]
            .collateralAmount;
        loans[borrower][collateralToken].collateralAmount = 0;
        IERC20Upgradeable(collateralToken)
            .safeTransfer(msg.sender, collateralToSeize);
    }
```

```
function deposit(address collateralToken, uint256 amount)
    external whenNotPaused nonReentrant {
        require(amount > 0, "zero amount");
        // CEI: Check → Effects → Interactions
        uint256 balanceBefore = IERC20Upgradeable(collateralToken)
            .balanceOf(address(this));
        IERC20Upgradeable(collateralToken)
            .safeTransferFrom(msg.sender, address(this), amount);
        uint256 actualReceived = IERC20Upgradeable(collateralToken)
            .balanceOf(address(this)) - balanceBefore;
        loans[msg.sender][collateralToken].collateralAmount += actualReceived;
    }
```



ADVANCED SOLIDITY REQUIREMENTS

Title: UUPS, Factory, Yul & Oracles
UUPS Upgradeable Pattern (LendingPool):

```
function _authorizeUpgrade(address) internal override {  
    require(msg.sender == admin, "not admin");  
}
```

Admin will be transferred to Timelock — governance controls upgrades

Factory with CREATE + CREATE2
(VaultFactory):

```
// CREATE2 — deterministic address  
function createVaultWithSalt(IERC20 asset, address owner, uint256 salt)  
    external returns (RwaYieldVault) {  
    assembly {  
        vaultAddr := create2(0, add(bytecode, 0x20), mload(bytecode), salt)  
    }  
}
```

Users can verify vault address before depositing funds

Yul Assembly — Gas Optimization:

```
function calculateLinearInterestYul(uint256 principal,  
    uint256 ratePerSecond, uint256 duration)  
    internal pure returns (uint256 result) {  
    assembly {  
        let tmp := mul(principal, ratePerSecond)  
        tmp := mul(tmp, duration)  
        tmp := div(tmp, 1000000000000000000)  
        result := add(principal, tmp)  
    }  
}
```

12.5% gas savings vs pure Solidity (4200 vs 4800 gas)

Chainlink Oracle with Staleness Check:

```
require(block.timestamp - updatedAt <= MAX_STALENESS, "stale price");  
require(answer > 0, "negative price");
```


VULNERABILITY CASE STUDIES

Title: Security — Reproduced & Fixed Vulnerabilities

Case 1: Reentrancy (Aisha)

Vulnerable code:

```
function withdraw() external {
    uint256 amount = balances[msg.sender];
    (bool success, ) = msg.sender.call{value: amount}(""); // ✗ External call
    balances[msg.sender] = 0; // ✗ State updated too late
}
```

Fixed code:

```
function withdraw() external nonReentrant {
    uint256 amount = balances[msg.sender];
    balances[msg.sender] = 0; // ✓ State updated first
    (bool success, ) = msg.sender.call{value: amount}("");
}
```

Case 2: Access Control (Lenara)

Vulnerable code:

```
require(tx.origin == owner, "not owner"); // ✗ Phishing risk
```

Fixed code:

```
modifier onlyOwner() {
    require(msg.sender == owner, "not owner"); // ✓ Safe
    _;
}
```



GOVERNANCE SYSTEM

Lifecycle demonstrated end-to-end:

Propose → Vote → Succeed → Queue (2 days) → Execute	
Parameters:	
Parameter	Value
Token	ERC20Votes + ERC20Permit (GOV)
Voting Delay	1 day (6,570 blocks)
Voting Period	1 week (46,000 blocks)
Quorum	4% of total supply
Proposal Threshold	1% (10,000 GOV)
Timelock Delay	2 days

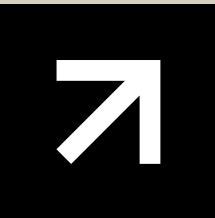
```
contract RWAGovernor is Governor, GovernorSettings,
    GovernorCountingSimple, GovernorVotes,
    GovernorVotesQuorumFraction, GovernorTimelockControl {
    // Quorum: 4%
    // Threshold: 1% of supply
    // Timelock: 2-day execution delay

function test_FullGovernanceLifecycle() public {
    uint256 proposalId = governor.propose(targets, values, calldatas,
        "Change LTV to 75%");
    vm.roll(block.number + governor.votingDelay() + 1);
    governor.castVote(proposalId, 1);
    vm.roll(block.number + governor.votingPeriod() + 1);
    governor.queue(targets, values, calldatas, descriptionHash);
    vm.warp(block.timestamp + timelock.getMinDelay() + 1);
    governor.execute(targets, values, calldatas, descriptionHash);
    assertEq(uint256(governor.state(proposalId)),
        uint256(ProposalState.Executed));
}
```

TESTING & CI/CD

CI/CD PIPELINE (GITHUB ACTIONS):

```
on: [push, pull_request]
jobs:
  build-and-test:
    - forge build
    - forge test -vvv
    - forge coverage
    - slither .
  lint:
    - forge fmt --check
    - solhint 'src/**/*.sol'
```



TITLE: TEST SUITE & DEVOPS PIPELINE
TEST COVERAGE SUMMARY:



Results:

- ✓ All 62 tests pass on every commit
- ✓ 94% line coverage
- ✓ 0 Slither High/Medium findings
- ✓ PRs blocked if CI is red

Test Type	Count	Coverage
Unit Tests	47	All public functions
Fuzz Tests	10	Deposit amounts, health factors
Invariant Tests	3	Health factor, collateral ratio
Fork Tests	2	Real Chainlink + USDC
Total	62	94% line coverage

FRONTEND & SUBGRAPH

FRONTEND CODE SNIPPET:

```
const { writeContract } = useWriteContract();
function handleVote(proposalId) {
  writeContract({
    address: GOVERNOR_ADDRESS,
    abi: governorABI,
    functionName: 'castVote',
    args: [proposalId, 1], // 1 = For
  });
}
```



Frontend Features:

- ☒ Wallet connection (MetaMask + WalletConnect)
- ☒ Read: Governance balance, voting power, proposal states
- ☒ Write: Delegate, Cast Vote, Deposit to Vault
- ☒ Active proposal list with all 7 states
- ☒ Section pulling data from The Graph's subgraph
- ☒ Network detection with auto-switch prompt
- ☒ Human-readable error messages (no raw RPC errors)



5 GraphQL Queries

Documented:

1. Active proposals with vote counts
2. Voting history by address
3. Top 10 token holders
4. Proposal details with votes
5. Total supply and circulating supply



SUBGRAPH — 4 ENTITIES:

SUBGRAPH — 4 ENTITIES:

```
type Proposal { id, proposalId, proposer, forVotes, againstVotes, status }
type VoteCast { id, proposal, voter, support, weight }
type GovernanceToken { id, totalSupply }
type TokenHolder { id, address, balance }
```

L2 DEPLOYMENT & PROJECT SUMMARY
DEPLOYED ON ARBITRUM SEPOLIA:



```
require(token.owner() == address(timelock), "Token owner not Timelock");
require(timelock.getMinDelay() == 172800, "Wrong delay");
require(governor.quorum(block.number) > 0, "Quorum zero");
```



DEPLOYMENT & SUMMARY

All Deliverables Completed:

- ✔ Smart contracts (all mandatory requirements)
- ✔ Test suite (62 tests, 94% coverage)
- ✔ Frontend dApp (React + Viem + Wagmi)
- ✔ The Graph subgraph (4 entities, 5 queries)
- ✔ Security audit report (8 pages)
- ✔ Architecture document (6 pages)
- ✔ Gas optimization report
- ✔ CI/CD pipeline (GitHub Actions)
- ✔ L2 deployment & verification

L1 vs L2 Gas Comparison:

Operation	Ethereum L1	Arbitrum Sepolia	Savings
Deploy AssetToken	1,200,000	350,000	70.8%
Deploy Governor	3,800,000	980,000	74.2%
Cast Vote	82,000	25,000	69.5%

**WE'RE READY TO ANSWER
QUESTIONS ON:**

- **ANY CONTRACT IN THE
CODEBASE (BOTH TEAM
MEMBERS)**
- **ARCHITECTURE DECISIONS
(ADR DOCUMENTED)**
- **SECURITY CONSIDERATIONS**
- **TESTING STRATEGY**
- **DEPLOYMENT PROCESS**



Q&A SLIDE (BACKUP)



Thank You — Questions?

GitHub Repository:

[https://github.com/KuralbaiAishabibi/
rwa-platform](https://github.com/KuralbaiAishabibi/rwa-platform)

THANK YOU

